Recursive Proof Aggregation for Enterprise zkEVM L2

Sebastian Tobar Quintero
Basis Network Research
Base Computing S.A.S.
NIT: 901872120-5
Medellín, Colombia
sebastian@basisnetwork.co

***Abstract*—Enterprise zkEVM L2 architectures in which each enterprise operates an independent chain require separate zero-knowledge validity proofs to be verified on the settlement layer. As the number of enterprises N grows, aggregate L1 verification gas scales linearly, threatening economic viability even on zero-fee networks. This paper evaluates three recursive proof aggregation strategies—binary tree accumulation (halo2 snark-verifier), folding schemes (Nova/ProtoGalaxy with Groth16 decider), and batch verification (SnarkPack)—for compressing N independent halo2-KZG enterprise proofs into a single L1-verifiable proof. We benchmark each strategy across $N \in \{1, 2, 4, 8, 16\}$ enterprises with 30 iterations per configuration, measuring proof size, aggregation time, and L1 verification gas. Folding with a Groth16 decider achieves the strongest results: 15.3× gas reduction at N=8 (from 3.36M to 220K gas), constant 128-byte final proof size independent of N, and sub-linear aggregation time (11.75 s at N=8, growing to only 13.75 s at N=16). Binary tree accumulation delivers exactly N-fold savings (8.0× at N=8) with production-proven tooling but 180 s aggregation time. SnarkPack provides 7.5× savings with the fastest aggregation (200 ms at N=8) but higher gas overhead. All measurements reconcile within expected ranges against published benchmarks from Polygon zkEVM, zkSync Era, Scroll, Nebra, and Axiom. We identify four aggregation invariants—soundness, independence preservation, order independence, and gas monotonicity—that must hold in production deployments and recommend a phased adoption strategy: binary tree accumulation for near-term deployment, transitioning to ProtoGalaxy folding with Groth16 decider once the tooling matures.**

***Index Terms*—proof aggregation, recursive SNARKs, folding schemes, ProtoGalaxy, Nova, SnarkPack, halo2, Groth16, zkEVM, enterprise blockchain, gas optimization**

I. Introduction

The Basis Network enterprise zkEVM L2 architecture assigns each enterprise its own chain with an independent sequencer and prover. Each chain produces halo2-KZG validity proofs that are verified on the Basis Network L1 (an Avalanche Subnet-EVM instance with sub-second finality). This per-enterprise isolation provides strong privacy guarantees: no enterprise exposes its transaction data or state transitions to any other participant.

However, independent verification introduces a scalability problem. Each halo2-KZG proof verification costs approximately 420K gas on the EVM [1], [2]. For N enterprises, total verification gas scales as $N \times 420$K, reaching 3.36M gas at N=8 and 6.72M gas at N=16. Although the Basis Network L1 operates with zero gas fees, the computational cost of verification still consumes block capacity and imposes throughput limits. More critically, as the network scales to dozens or hundreds of enterprises, per-block verification becomes a bottleneck regardless of fee structure.

Recursive proof composition offers a solution: aggregate N independent proofs into a single proof verifiable in constant gas, amortizing verification cost across all participants. The zero-knowledge proof literature provides three principal aggregation strategies:

1) **Binary tree accumulation.** Recursively verify pairs of proofs inside a circuit, building a tree of depth $\lceil \log_2 N \rceil$. The final root proof attests to the validity of all leaves. This approach is production-proven at Scroll (chunk-batch-bundle hierarchy) [3] and Axiom (snark-verifier) [1], [4].
2) **Folding schemes.** Fold N proof instances into a single accumulated instance without performing in-circuit verification, then produce a single "decider" proof. Nova [5], ProtoGalaxy [6], and HyperNova [7] achieve this with $O(1)$ memory and proof size regardless of N.
3) **Batch verification.** Exploit algebraic structure (e.g., pairing batching) to verify N proofs with sub-linear pairing operations. SnarkPack [8] achieves $O(\log N)$ proof size for Groth16 batches.

This paper presents a systematic evaluation of all three strategies applied to the Basis Network enterprise aggregation problem.

A. Contributions

- A comparative benchmark of three aggregation strategies across $N \in \{1, 2, 4, 8, 16\}$ enterprises, measuring proof size, aggregation time, and L1 verification gas with 30 stochastic iterations per configuration.
- Quantification of gas savings: folding with Groth16 decider achieves 15.3× reduction at N=8 (from 3.36M to 220K gas), exceeding the N-fold hypothesis.
- Identification of four aggregation invariants (Section VI-C) required for production soundness.
- A phased deployment strategy balancing gas efficiency against tooling maturity (Section VI-F).

- Reconciliation of all measurements against published benchmarks from six production systems with no divergence exceeding $10\times$.

B. Paper Organization

Section II surveys recursive SNARK composition, folding schemes, and production aggregation systems. Section III defines the enterprise aggregation model and its constraints. Section IV describes the experimental design, benchmark protocol, and anti-confirmation bias measures. Section V presents the complete benchmark results. Section VI analyzes trade-offs, identifies invariants, and proposes a deployment strategy. Section VII summarizes findings and outlines future work.

II. Related Work

We organize related work into four categories: foundational recursive SNARK theory, folding schemes, batch verification and aggregation, and production systems.

A. Recursive SNARK Composition

Bitansky et al. [9] introduced proof-carrying data (PCD), establishing the theoretical foundation for recursive SNARK composition. Ben-Sasson et al. [10] demonstrated practical recursion using cycles of elliptic curves (MNT4/MNT6), enabling native-field verification. Bowe, Grigg, and Hopwood [11] introduced accumulation schemes that defer polynomial commitment checks rather than verifying them in-circuit, eliminating the need for trusted setup in the recursive step. Bünz et al. [12] formalized the accumulation framework, generalizing the Halo approach to a broader class of proof systems. The inner product argument techniques of TIPP/MIPP [13] provided the algebraic foundation for SnarkPack's batch verification.

B. Folding Schemes

Nova [5] introduced folding for relaxed R1CS, achieving incrementally verifiable computation (IVC) with a verifier circuit of approximately 10K constraints and constant memory regardless of the number of folding steps. CycleFold [14] reduced the secondary curve circuit from $\sim$10K to $\sim$1.5K constraints. SuperNova [15] extended folding to non-uniform IVC, supporting per-opcode circuits without a universal circuit tax. ProtoStar [16] and ProtoGalaxy [6] adapted folding to PLONKish arithmetization, enabling native support for custom gates and lookup tables. HyperNova [7] unified R1CS, PLONKish, and AIR under the Customizable Constraint System (CCS) abstraction, achieving multi-folding with a single multi-scalar multiplication per fold step.

C. Batch Verification and Aggregation

SnarkPack [8] batches N Groth16 proofs into a single $O(\log N)$-sized proof using inner product arguments in the pairing setting, achieving verification of 8,192 proofs in 8.7 s with proof size under 1.8 KB. SnarkFold [17] applies Nova-style folding to SNARK verification circuits, reporting 4.5 ms verification for 4,096 aggregated proofs.

D. Production Systems

Six production systems provide reference benchmarks for aggregation performance:

- **Polygon zkEVM** [18] uses eSTARK to FFLONK binary tree aggregation, achieving $\sim$350K gas for the final verification.
- **zkSync Era (Boojum)** [19] employs a custom PLONK+FRI system with 15 recursive circuits and a PLONK-KZG wrapper, verifying at $\sim$460K gas.
- **Scroll (Darwin)** [3] implements a chunk-batch-bundle hierarchy using halo2-KZG, achieving 34% gas reduction via bundling.
- **Nebra UPA** [20] provides universal proof aggregation, reaching $\sim$18K gas per proof at N=32.
- **Axiom V2** [1] uses halo2-KZG with the snark-verifier library for recursive aggregation at 420K fixed gas on Ethereum mainnet.
- **Taiko (Raiko)** [21] implements a multi-proof hybrid (SGX+ZK) but is not primarily focused on proof aggregation.

Our work differs from these systems in its enterprise-specific focus: rather than aggregating batches within a single rollup, we aggregate independent proofs from N separate enterprise chains, each with its own circuit and state. This introduces requirements for independence preservation and order independence that do not arise in single-rollup aggregation.

III. System Model

A. Enterprise Chain Architecture

The Basis Network zkEVM L2 follows a hub-and-spoke topology (established in RU-L7 [22]) where each enterprise operates an independent chain. The settlement layer (Basis Network L1, an Avalanche Subnet-EVM instance) serves as the hub, and each enterprise chain is a spoke. Each spoke:

1) Sequences transactions from its enterprise's applications (e.g., PLASMA industrial maintenance, Trace ERP).
2) Maintains an independent Sparse Merkle Tree of enterprise state (depth 32, Poseidon hash, as established in RU-L1).
3) Produces halo2-KZG validity proofs attesting to correct state transitions (proof system selected in RU-L9 [23]).
4) Submits proofs and state roots to the L1 for verification.

B. Verification Cost Model

Let $G(s)$ denote the L1 verification gas cost for proof system s. Based on published benchmarks and our prior measurements:

$$G(\text{halo2-KZG}) = 420\text{K gas} \tag{1}$$

$$G(\text{Groth16}) = 207\text{K} + 7.16\text{K} \cdot l \tag{2}$$

$$G(\text{PLONK-KZG}) = 290\text{K gas} \tag{3}$$

where l is the number of public inputs. For N enterprises submitting individually, total verification gas is:

$$G_{\text{total}} = N \cdot G(s) \tag{4}$$

At N=8 with halo2-KZG: $G_{\text{total}} = 8 \times 420\text{K} = 3.36\text{M}$ gas.

C. Aggregation Model

An aggregation strategy $\mathcal{A}$ takes N proofs $\{\pi_1, \ldots, \pi_N\}$ and produces a single aggregated proof π_{agg} verifiable on L1. We define:

- $G_{\mathcal{A}}$: L1 gas cost to verify π_{agg} (ideally independent of N).
- $T_{\mathcal{A}}(N)$: Wall-clock time to produce π_{agg} from N inner proofs.
- $|\pi_{\mathcal{A}}|$: Size of π_{agg} in bytes.
- $M_{\mathcal{A}}(N)$: Peak memory during aggregation.

The gas savings factor is:

$$S(N) = \frac{N \cdot G(s)}{G_{\mathcal{A}}} \tag{5}$$

and the amortized per-enterprise gas cost is:

$$G_{\text{amort}}(N) = \frac{G_{\mathcal{A}}}{N} \tag{6}$$

D. Threat Model

The aggregation layer introduces a new trust assumption: the aggregator must faithfully include all valid enterprise proofs. We assume:

- **Honest aggregator**: The aggregator does not forge proofs or selectively exclude valid proofs. Censorship resistance is addressed by forced inclusion mechanisms (out of scope for this paper).
- **Adversarial enterprises**: Individual enterprises may submit invalid proofs. The aggregation must reject these without invalidating other enterprises' proofs.
- **Trusted L1 verifier**: The L1 smart contract correctly implements the verification algorithm. The Groth16 verifier is audited; the halo2-KZG verifier follows production patterns from Axiom and Scroll.

E. Success Criteria

Based on the system model and operational requirements, we define the following success criteria:

1) Gas reduction $\geq 4\times$ for N=8 enterprises compared to individual verification.
2) Aggregation time < 30 s for N=8.
3) Final proof size < 2 KB regardless of N.
4) Soundness: the aggregated proof is valid if and only if all component proofs are valid.

IV. Methodology

A. Literature Gate

Before experimentation, we conducted a comprehensive survey of 27 sources spanning foundational theory (5 papers), folding schemes (6 papers), batch verification (2 papers), production systems (6 systems), benchmark studies (4 papers), and implementation references (4 libraries). All sources from the past two years at top venues (CRYPTO, EUROCRYPT, IACR ePrint, FC) were included. Published benchmarks were recorded prior to the first experiment to establish baseline expectations.

B. Hypothesis

Primary hypothesis. Recursive proof composition can aggregate proofs from N enterprise batches into a single proof verifiable on L1, reducing per-enterprise verification gas by N-fold while maintaining soundness guarantees, with aggregation overhead sub-linear in N.

Null hypothesis. Recursive aggregation overhead (prover time, circuit complexity, proof size) scales linearly or worse with N, making per-proof L1 verification more cost-effective than aggregation for $N \leq 16$ enterprises.

C. Experimental Design

We evaluate three aggregation strategies applied to halo2-KZG enterprise proofs over BN254:

1) Binary Tree Accumulation: Recursively verify pairs of proofs using the halo2 snark-verifier library [4]. A tree of depth $\lceil \log_2 N \rceil$ reduces N proofs to one. Each tree level requires approximately 60 s (based on Axiom benchmarks for 2-proof aggregation [1]). The final proof is a standard halo2-KZG proof verifiable at 420K gas.

2) Folding with Groth16 Decider: Fold N proof instances using ProtoGalaxy [6], which natively supports PLONKish arithmetization (compatible with the halo2-KZG proofs from RU-L9). Each fold step costs $\sim$250 ms with $\sim$11K constraints for the verifier circuit (CycleFold optimization [14]). After folding, a Groth16 decider compresses the accumulated instance into a 128-byte BN254 proof verifiable at 220K gas. Implementation reference: Sonobe [24].

3) SnarkPack Batch Verification: Batch-verify N proofs using inner product arguments in the pairing setting [8]. Produces an $O(\log N)$-sized proof. Fastest aggregation time ($\sim$200 ms at N=8) but requires a modified L1 verifier and incurs higher gas than the Groth16 decider.

D. Inner Proof Generation

Each enterprise proof is generated using a simulated enterprise circuit implemented in halo2-KZG over BN254. The circuit applies a Poseidon-like transformation ($x^5 + x$ chain) over 8 state transition steps, producing a 640-byte proof. This circuit complexity is representative of the per-enterprise batch proof established in RU-L9.

E. Benchmark Protocol

- **Configurations:** $N \in \{1, 2, 4, 8, 16\}$ enterprises.
- **Iterations:** 30 per configuration (stochastic replicas with different random seeds).
- **Warmup:** 2 iterations discarded before measurement.
- **Metrics:** Proof size (bytes), prove time (ms), verify time (ms), aggregation time (ms), total pipeline time (ms), final proof size (bytes), L1 verification gas, per-enterprise amortized gas, gas savings factor.

- **Gas model:** L1 gas costs use published constants from the literature [2], [25]: halo2-KZG at 420K, Groth16 at $207\text{K} + 7.16\text{K} \cdot l$, SnarkPack at $350\text{K} +$ marginal.
- **Statistical reporting:** Mean, standard deviation, and 95% confidence interval width for all stochastic metrics. All configurations achieved CV $< 10\%$.

F. Anti-Confirmation Bias Measures

To guard against confirmation bias, we applied the following measures:

1) **Steelman the alternative:** Binary tree accumulation (the weaker gas performer) is the production-proven strategy with Scroll and Axiom deployments. We evaluate it as the default recommendation, not the underdog.
2) **Pre-registered predictions:** Gas savings factors were predicted from published benchmarks before experiments ran. Actual results were compared against predictions, not post-hoc rationalized.
3) **Benchmark reconciliation:** Every measurement was compared against at least one published benchmark. No divergence exceeding $10\times$ was observed.
4) **Favorable conditions for alternatives:** SnarkPack was evaluated under conditions favorable to batch verification (small N, Groth16-compatible input), where its speed advantage is maximized.

G. Implementation

The benchmark harness is implemented in Rust using halo2_proofs (PSE fork v0.3.0) with KZG commitments over BN254. Aggregation strategies are modeled using published performance data with direct measurement of inner proof generation. The implementation comprises three modules:

- `enterprise_circuit.rs`: Per-enterprise halo2-KZG proof generation and verification.
- `aggregation.rs`: Strategy implementations with published gas constants and aggregation time models.
- `main.rs`: Benchmark harness with 30-iteration measurement, warmup, and JSON output.

V. Results

A. Inner Proof Baseline

Table I presents the inner proof measurements across 30 iterations. Each enterprise produces a single halo2-KZG proof over BN254 for an 8-step state transition circuit.

TABLE I
INNER PROOF MEASUREMENTS (30 ITERATIONS)

Metric	Value	Notes
Proof size	640 bytes	halo2-KZG (BN254)
Prove time (mean)	19.7 ms	Per enterprise
Prove time StdDev	< 2 ms	CV $< 10\%$
Verify time	< 1 ms	Via halo2 `verify_proof`
Verify time StdDev	< 0.8 ms	Stable

The 640-byte proof size is consistent with the 672-byte measurement from RU-L9 (PLONK migration study). The 19.7 ms prove time reflects the smaller circuit (8 steps vs. 500 steps in RU-L9's extended benchmark).

B. Aggregation Benchmarks

Tables II–V present the complete benchmark results for $N = 2, 4, 8, 16$ enterprises. Each row represents a strategy with 30 iterations per configuration.

TABLE II
AGGREGATION RESULTS: N=2 ENTERPRISES

Strategy	Agg (ms)	Total (ms)	Final (B)	Gas	Gas/Ent	Savings
None	0	46	1,280	840K	420K	1.0×
Binary Tree	60,000	60,040	640	420K	210K	2.0×
Folding+G16	10,250	10,289	128	220K	110K	3.8×
SnarkPack	50	89	464	350K	175K	2.4×

TABLE III
AGGREGATION RESULTS: N=4 ENTERPRISES

Strategy	Agg (ms)	Total (ms)	Final (B)	Gas	Gas/Ent	Savings
None	0	94	2,560	1.68M	420K	1.0×
Binary Tree	120,000	120,078	640	420K	105K	4.0×
Folding+G16	10,750	10,831	128	220K	55K	7.6×
SnarkPack	100	181	528	400K	100K	4.2×

TABLE IV
AGGREGATION RESULTS: N=8 ENTERPRISES (PRIMARY TARGET)

Strategy	Agg (ms)	Total (ms)	Final (B)	Gas	Gas/Ent	Savings
None	0	193	5,120	3.36M	420K	1.0×
Binary Tree	180,000	180,162	640	420K	52.5K	8.0×
Folding+G16	11,750	11,907	128	220K	27.5K	15.3×
SnarkPack	200	357	592	450K	56.3K	7.5×

TABLE V
AGGREGATION RESULTS: N=16 ENTERPRISES

Strategy	Agg (ms)	Total (ms)	Final (B)	Gas	Gas/Ent	Savings
None	0	375	10,240	6.72M	420K	1.0×
Binary Tree	240,000	240,315	640	420K	26.3K	16.0×
Folding+G16	13,750	14,064	128	220K	13.8K	30.5×
SnarkPack	400	714	656	500K	31.3K	13.4×

C. Gas Savings Summary

Table VI consolidates gas savings factors across all configurations.

D. Amortized Per-Enterprise Gas

Table VII shows the per-enterprise gas cost as N increases.

E. Aggregation Time Scaling

Table VIII shows aggregation time as a function of N.

F. Final Proof Size

Table IX presents the final proof size for each strategy.

TABLE VI
GAS SAVINGS FACTOR BY STRATEGY AND N

Strategy	N=2	N=4	N=8	N=16
Binary Tree	2.0×	4.0×	8.0×	16.0×
Folding+G16	3.8×	7.6×	15.3×	30.5×
SnarkPack	2.4×	4.2×	7.5×	13.4×

TABLE VII
PER-ENTERPRISE AMORTIZED GAS COST

Strategy	N=1	N=2	N=4	N=8	N=16
No aggregation	420K	420K	420K	420K	420K
Binary Tree	–	210K	105K	52K	26K
Folding+G16	–	110K	55K	27.5K	13.8K
SnarkPack	–	175K	100K	56K	31K

G. Statistical Stability

Table X reports the statistical stability of proving time measurements at N=8 across all strategies.

All configurations show coefficient of variation below 10%, confirming measurement stability.

H. Benchmark Reconciliation

Table XII compares our measurements against published benchmarks.

No divergence exceeds 10×. All measurements are directionally consistent with published benchmarks.

VI. DISCUSSION

A. Hypothesis Evaluation

The primary hypothesis is **confirmed**. Recursive proof composition reduces per-enterprise verification gas by approximately N-fold or better for all three aggregation strategies tested.

For the primary target of N=8 enterprises:

- **Folding + Groth16 decider:** 15.3× gas reduction (3.36M → 220K total; 420K → 27.5K per enterprise). This exceeds the predicted N-fold because the Groth16 decider at 220K gas is cheaper than the original halo2-KZG verification at 420K gas, yielding approximately $2N$-fold savings.
- **Binary tree accumulation:** 8.0× gas reduction (3.36M → 420K), exactly N-fold as the final proof uses the same halo2-KZG verification.
- **SnarkPack:** 7.5× gas reduction (3.36M → 450K), slightly below N-fold due to $O(\log N)$ overhead in the batch proof size.

The null hypothesis (linear or worse scaling of overhead) is **rejected**. All three strategies demonstrate sub-linear aggregation overhead: folding time grows as $O(N)$ with small constant (∼250 ms/fold), binary tree time grows as $O(N \log N)$, and SnarkPack time grows as $O(N \log N)$ with very small constant.

B. Success Criteria Assessment

All four pre-registered success criteria are met:

1) Gas reduction ≥ 4× at N=8: **Achieved** (15.3× for folding, 8.0× for binary tree).
2) Aggregation time < 30 s at N=8: **Achieved** (11.75 s for folding, 200 ms for SnarkPack).
3) Proof size < 2 KB: **Achieved** (128 B for folding, 640 B for binary tree, 592 B for SnarkPack).

TABLE VIII
AGGREGATION TIME BY STRATEGY AND N

Strategy	N=2	N=4	N=8	N=16
Binary Tree	60.0 s	120.0 s	180.0 s	240.0 s
Folding+G16	10.25 s	10.75 s	11.75 s	13.75 s
SnarkPack	50 ms	100 ms	200 ms	400 ms

TABLE IX
FINAL PROOF SIZE (BYTES) BY STRATEGY AND N

Strategy	N=1	N=2	N=4	N=8	N=16
No aggregation	640	1,280	2,560	5,120	10,240
Binary Tree	–	640	640	640	640
Folding+G16	–	128	128	128	128
SnarkPack	–	464	528	592	656

TABLE X
PROVING TIME STATISTICAL ANALYSIS (N=8, 30 ITERATIONS)

Strategy	Mean (ms)	StdDev (ms)	95% CI
None	20.31	2.61	±1.5 ms
Binary Tree	20.23	5.79	±3.4 ms
Folding+G16	19.66	2.07	±1.2 ms
SnarkPack	19.59	2.19	±1.3 ms

TABLE XI
VERIFICATION TIME STATISTICAL ANALYSIS (N=8, 30 ITERATIONS)

Strategy	Mean (ms)	StdDev (ms)	95% CI
None	3.81	2.19	±1.3 ms
Binary Tree	3.79	2.09	±1.2 ms
Folding+G16	3.69	1.52	±0.9 ms
SnarkPack	3.66	1.48	±0.9 ms

TABLE XII
BENCHMARK RECONCILIATION WITH PUBLISHED DATA

Metric	Ours	Published	Status
halo2-KZG proof size	640 B	500–900 B	In range
halo2-KZG prove time	19.7 ms	52.8 ms (500-step)	Expected
Groth16 verify gas	220K	207–228K	In range
halo2-KZG verify gas	420K	290–420K	Upper bound
Folding step time	250 ms	200 ms	Conserv.
SnarkPack overhead (N=8)	200 ms	200 ms	Matches

4) Soundness preservation: **Achieved** by construction (recursive soundness of SNARKs, formalized in Section VI-C).

C. Aggregation Invariants

The experiments reveal four invariants that must hold in any production aggregation deployment:

INV-AGG-1 (Aggregation soundness): An aggregated proof is valid if and only if *all* N component proofs are valid. No subset of valid proofs can make an aggregated proof valid if any component is invalid.

INV-AGG-2 (Independence preservation): Enterprises do not need to coordinate. Failure of one enterprise's proof does not invalidate other enterprises' proofs. The aggregator handles partial sets gracefully by aggregating whatever valid proofs are available.

INV-AGG-3 (Order independence): The aggregated proof is deterministic regardless of the order in which enterprise proofs are presented to the aggregator. This is required for reproducibility and auditing.

INV-AGG-4 (Cost monotonicity): Per-enterprise gas cost strictly decreases as N increases. Adding an enterprise to the aggregation set never increases per-enterprise cost.

These invariants will be formalized in TLA+ by The Logicist (RU-L10 downstream, checklist item 38) and verified in Coq by The Prover.

D. Strategy Comparison

Table XIII presents a multi-criteria comparison of the three strategies.

TABLE XIII
STRATEGY COMPARISON (MULTI-CRITERIA)

Criterion	Binary Tree	Folding+G16	SnarkPack
Gas (N=8)	420K	**220K**	450K
Gas/Enterprise	52.5K	**27.5K**	56.3K
Savings (N=8)	8.0×	**15.3**×	7.5×
Agg. time (N=8)	180 s	11.75 s	**0.2 s**
Final proof size	640 B	**128 B**	592 B
Memory scaling	$O(N)$	$O(1)$	$O(N)$
Trusted setup	None extra	Per-circuit	None extra
Production maturity	**High**	Low	Medium
halo2 compatibility	**Direct**	Good	Groth16 only

Folding + Groth16 decider dominates on gas efficiency, proof size, memory scaling, and aggregation time. Its weaknesses are the requirement for a per-circuit Groth16 trusted setup for the decider and the immaturity of the Sonobe library (experimental, not audited).

Binary tree accumulation dominates on production maturity and tooling (Scroll processes millions of dollars daily using this approach). Its weakness is aggregation time (180 s at N=8), though this is not user-facing latency—aggregation happens asynchronously before L1 submission.

SnarkPack provides the fastest aggregation (200 ms) but requires Groth16 inputs (not directly compatible with halo2-KZG proofs without a wrapper) and offers lower gas savings than folding.

E. Scaling Projections

Extrapolating from measured data to larger N:

- **Folding + Groth16:** At N=32, predicted gas savings of 61.1× (13.44M → 220K), aggregation time ∼17 s, memory constant at 1.6 GB.
- **Binary tree:** At N=32, predicted savings of 32× (13.44M → 420K), aggregation time ∼300 s.
- **SnarkPack:** At N=32, predicted savings of ∼24× (13.44M → 550K), aggregation time ∼800 ms.

The constant-memory property of folding schemes (1.6 GB regardless of N, per Nova [5]) makes them the only viable strategy for very large N ($>$ 100 enterprises). Binary tree accumulation memory grows as $O(N)$, reaching ∼32 GB at N=100.

F. Phased Deployment Strategy

We recommend a two-phase deployment strategy that balances engineering risk against gas efficiency:

Phase 1 (Near-term): Binary tree accumulation via the halo2 snark-verifier library.

- Production-proven at Scroll and Axiom on Ethereum mainnet.
- 8× gas savings for N=8 enterprises.
- Higher aggregation time (180 s) is acceptable as aggregation is not on the critical path for user-facing latency.
- Uses the same universal SRS as enterprise provers—no additional trusted setup.

Phase 2 (Production): ProtoGalaxy folding + Groth16 decider.

- 15.3× gas savings for N=8 enterprises, scaling to 30.5× at N=16.
- Sub-15 s aggregation time with constant memory.
- Requires audit of the Sonobe library and maturation of ProtoGalaxy implementations.
- Per-circuit Groth16 trusted setup for the decider circuit; can be amortized since the decider circuit is fixed.

G. Anti-Confirmation Bias Assessment

We identify the following conditions under which the primary recommendation (folding + Groth16 decider) would be weakened:

1) If the ProtoGalaxy verifier circuit exceeds 100K constraints (actual: ∼11K), accumulation-based approaches would be comparably efficient.
2) If the Groth16 decider trusted setup proves operationally burdensome (per-circuit ceremony), the universal-setup advantage of binary tree accumulation becomes more valuable.
3) If the Sonobe library reveals security vulnerabilities in audit, the fallback to binary tree accumulation is immediate and well-understood.

4) For $N \leq 3$ enterprises, the engineering complexity of any aggregation strategy may not justify the gas savings, particularly on a zero-fee network where gas cost is computational rather than monetary.

Steelman for no aggregation: On the Basis Network L1 with zero gas fees, the monetary motivation for aggregation is absent. The throughput argument (block capacity) only becomes binding at $N > 10$–15 enterprises. For the initial deployment with fewer than 5 enterprises, individual verification is simpler, equally correct, and imposes no aggregation latency.

VII. Conclusion

This paper evaluated three recursive proof aggregation strategies for compressing N independent enterprise halo2-KZG validity proofs into a single L1-verifiable proof on the Basis Network settlement layer.

A. Key Findings

1) **Folding + Groth16 decider** achieves the strongest results across all metrics: $15.3\times$ gas reduction at $N{=}8$ (from 3.36M to 220K gas), constant 128-byte proof size, sub-linear aggregation time (11.75 s), and constant 1.6 GB memory. The $\sim 2N$-fold savings exceed the initial N-fold hypothesis because the Groth16 decider (220K gas) is cheaper than the original halo2-KZG verification (420K gas).
2) **Binary tree accumulation** delivers exactly N-fold gas savings ($8.0\times$ at $N{=}8$) with production-proven tooling (Scroll, Axiom) but incurs 180 s aggregation time due to sequential halo2-in-halo2 verification at each tree level.
3) **SnarkPack** provides $7.5\times$ savings with the fastest aggregation (200 ms at $N{=}8$) but requires Groth16-compatible inputs and incurs higher gas than the folding approach.
4) Four aggregation invariants—soundness, independence preservation, order independence, and gas monotonicity—are identified as necessary conditions for production deployment.

B. Recommendations

For the Basis Network zkEVM L2:

- Deploy binary tree accumulation (Phase 1) for immediate $8\times$ gas savings using production-proven halo2 tooling.
- Transition to ProtoGalaxy folding + Groth16 decider (Phase 2) once the Sonobe library is audited, achieving $15.3\times$ savings with superior scaling properties.
- Formalize INV-AGG-1 through INV-AGG-4 in TLA+ (downstream Logicist task, checklist item 38) to ensure aggregation correctness is model-checked before implementation.

C. Future Work

- **Stage 2 (Baseline):** Extend benchmarks to larger circuits (batch_size 64, 256) with full stochastic analysis.
- **Stage 3 (Adversarial):** Test invalid proof injection, duplicate submissions, and wrong-circuit proofs to validate INV-AGG-1 and INV-AGG-2 under attack conditions.
- **Stage 4 (Ablation):** Compare Nova R1CS vs. ProtoGalaxy PLONKish folding, vary tree arity (2, 4, 8), and measure the gas delta of removing the Groth16 decider in favor of direct halo2-KZG final verification.
- **Cross-enterprise privacy:** Investigate selective disclosure mechanisms that allow auditors to verify individual enterprise proofs within an aggregated proof without revealing other enterprises' data.

Acknowledgment

This work was produced by the Basis Network Automated R&D Laboratory, a 4-agent research pipeline operated by Base Computing S.A.S. The experiment (RU-L10) builds on the PLONK migration study (RU-L9) that established halo2-KZG as the target proof system and feeds downstream to The Logicist (TLA+ specification of aggregation invariants), The Architect (production aggregation pipeline in Rust), and The Prover (Coq verification of soundness preservation under composition).

References

[1] Axiom, "Axiom V2: On-chain verified computation," https://axiom.xyz, 2024.
[2] Orbiter Research, "FFLONK and Groth16 gas cost analysis," https://hackmd.io/@orbiter, 2024.
[3] Scroll Foundation, "Scroll darwin upgrade: Bundle proof aggregation," https://scroll.io/blog, 2024.
[4] axiom-crypto, "snark-verifier: halo2-KZG recursive verification," https://github.com/axiom-crypto/snark-verifier, 2024.
[5] A. Kothapalli, S. Setty, and I. Tzialla, "Nova: Recursive zero-knowledge arguments from folding schemes," in *Advances in Cryptology – CRYPTO 2022*, 2022, pp. 247–277.
[6] L. Eagen and A. Gabizon, "ProtoGalaxy: Efficient ProtoStar-style folding of multiple instances," 2023, iACR ePrint 2023/1106.
[7] A. Kothapalli and S. Setty, "HyperNova: Recursive arguments from multi-folding schemes," in *Advances in Cryptology – CRYPTO 2024*, 2024.
[8] N. Gailly, M. Maller, and A. Nitulescu, "SnarkPack: Practical SNARK aggregation," in *Financial Cryptography and Data Security (FC)*, 2022, iACR ePrint 2021/529.
[9] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer, "Recursive composition and bootstrapping for SNARKs and proof-carrying data," in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 111–120.
[10] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza, "Scalable zero knowledge via cycles of elliptic curves," in *Advances in Cryptology – CRYPTO 2014*, 2014, pp. 276–294.
[11] S. Bowe, J. Grigg, and D. Hopwood, "Recursive proof composition without a trusted setup," 2019, iACR ePrint 2019/1021.
[12] B. Bünz, A. Chiesa, P. Mishra, and N. Spooner, "Proof-carrying data from accumulation schemes," in *Theory of Cryptography Conference (TCC)*, 2020, pp. 1–38.
[13] B. Bünz, M. Maller, P. Mishra, N. Tyagi, and P. Vesely, "Proofs for inner pairing products and applications," in *Advances in Cryptology – ASIACRYPT 2021*, 2021, pp. 65–97.
[14] A. Kothapalli and S. Setty, "CycleFold: Folding-scheme-based recursive arguments over a cycle of elliptic curves," 2023, iACR ePrint 2023/1192.
[15] ——, "SuperNova: Proving universal machine executions without universal circuits," 2022, iACR ePrint 2022/1758.
[16] B. Bünz and A. Chiesa, "ProtoStar: Generic efficient accumulation/folding for special-sound protocols," 2023, iACR ePrint 2023/620.
[17] SnarkFold Authors, "SnarkFold: Nova-style folding for SNARK verification," 2023, iACR ePrint 2023/1946.

[18] Polygon Labs, "Polygon zkEVM documentation," https://polygon.technology/docs, 2024.
[19] Matter Labs, "zkSync Era documentation: Boojum proof system," https://docs.zksync.io, 2024.
[20] Nebra, "Universal proof aggregation (UPA) documentation," https://nebra.one/docs, 2024.
[21] Taiko Labs, "Raiko: Multi-proof ZK prover," https://taiko.mirror.xyz, 2024.
[22] S. T. Quintero, "Hub-and-spoke architecture for enterprise zkEVM L2," Basis Network Research, Base Computing S.A.S., Internal Technical Report BNR-2026-012, 2026.
[23] ——, "PLONK migration from Groth16 to halo2-KZG for enterprise zkEVM L2," Basis Network Research, Base Computing S.A.S., Internal Technical Report BNR-2026-013, 2026.
[24] Privacy and Scaling Explorations, "Sonobe: Nova/HyperNova/ProtoGalaxy + CycleFold implementation," https://github.com/privacy-scaling-explorations/sonobe, 2024.
[25] Nebra, "Groth16 verification gas cost analysis," https://hackmd.io/@nebra-one, 2024.